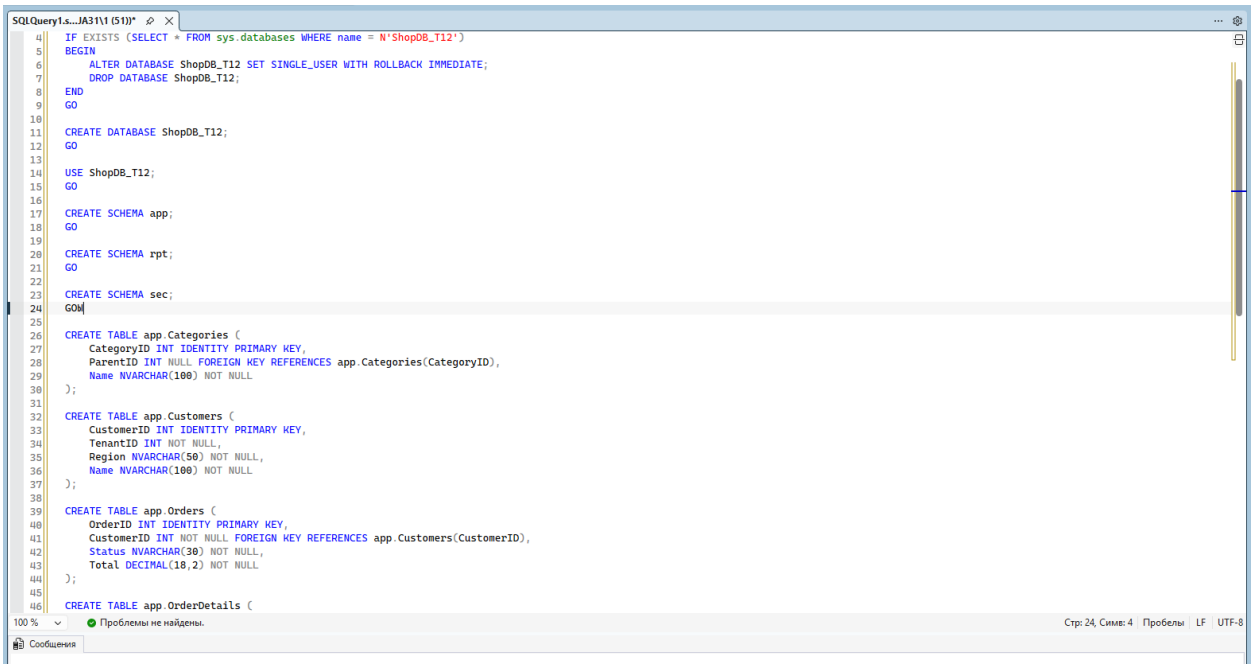
Стружинский Александр Сегреевич

Подготовка к работе, всё что требутся для моего 12 билета, я реализовал



```
IF EXISTS (SELECT * FROM sys.databases WHERE name = N'ShopDB_T12')
BEGIN
    ALTER DATABASE ShopDB_T12 SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE ShopDB_T12;
END
GO

CREATE DATABASE ShopDB_T12;
GO

USE ShopDB_T12;
GO

CREATE SCHEMA app;
GO

CREATE SCHEMA rpt;
GO

CREATE SCHEMA sec;
GO

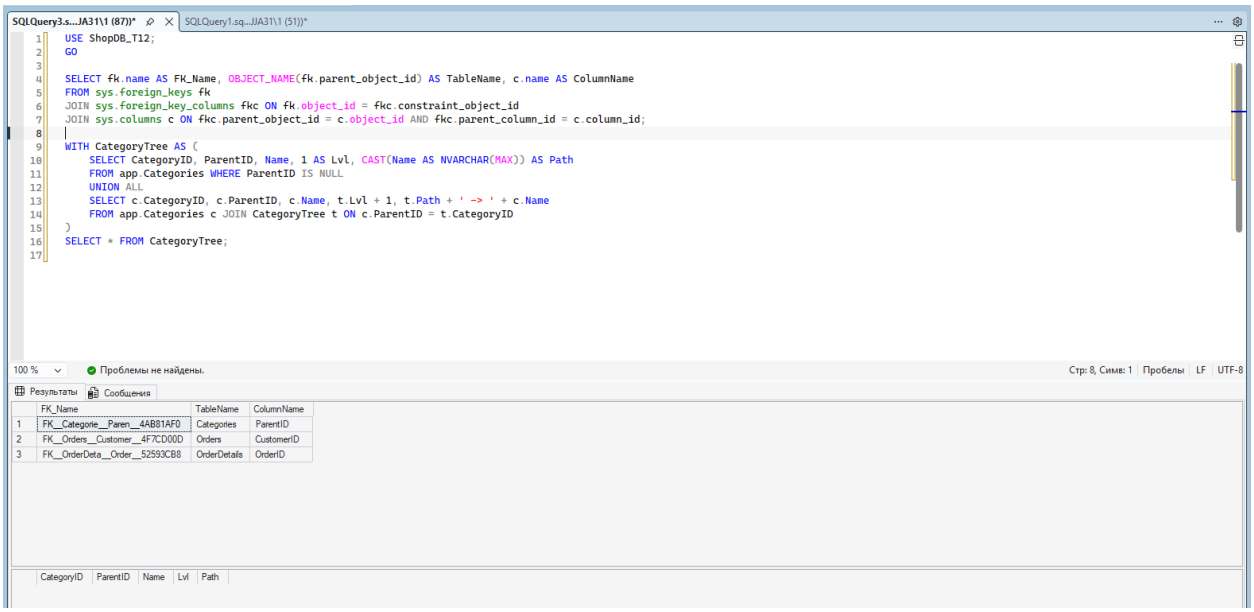
CREATE TABLE app.Categories (
    CategoryID INT IDENTITY PRIMARY KEY,
    ParentID INT NULL FOREIGN KEY REFERENCES app.Categories(CategoryID),
    Name NVARCHAR(100) NOT NULL
);

CREATE TABLE app.Customers (
    CustomerID INT IDENTITY PRIMARY KEY,
    TenantID INT NOT NULL,
    Region NVARCHAR(50) NOT NULL,
    Name NVARCHAR(100) NOT NULL
);

CREATE TABLE app.Orders (
    OrderID INT IDENTITY PRIMARY KEY,
    CustomerID INT NOT NULL FOREIGN KEY REFERENCES app.Customers(CustomerID),
    Status NVARCHAR(30) NOT NULL,
    Total DECIMAL(18,2) NOT NULL
);

CREATE TABLE app.OrderDetails (
```

1 Задание



```
USE ShopDB_T12;
GO

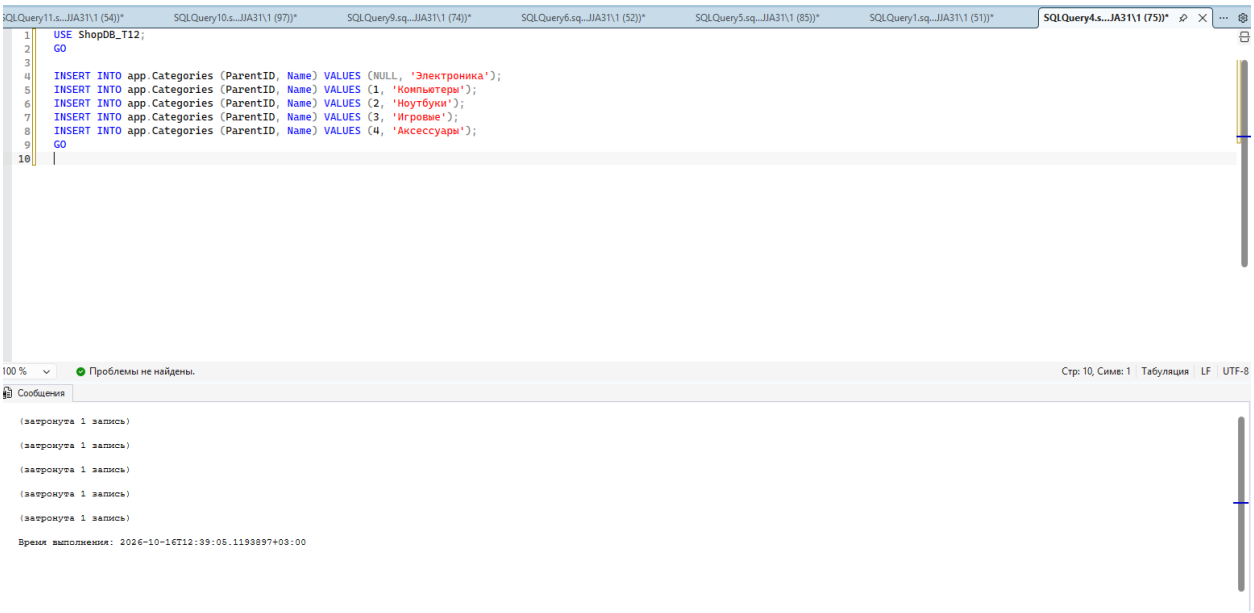
SELECT fk.name AS FK_Name, OBJECT_NAME(fk.parent_object_id) AS TableName, c.name AS ColumnName
FROM sys.foreign_keys fk
JOIN sys.foreign_key_columns fkc ON fk.object_id = fkc.constraint_object_id
JOIN sys.columns c ON fkc.parent_object_id = c.object_id AND fkc.parent_column_id = c.column_id;

WITH CategoryTree AS (
    SELECT CategoryID, ParentID, Name, 1 AS Lvl, CAST(Name AS NVARCHAR(MAX)) AS Path
    FROM app.Categories WHERE ParentID IS NULL
    UNION ALL
    SELECT c.CategoryID, c.ParentID, c.Name, t.Lvl + 1, t.Path + ' -> ' + c.Name
    FROM app.Categories c JOIN CategoryTree t ON c.ParentID = t.CategoryID
)
SELECT * FROM CategoryTree;
```

	FK_Name	TableName	ColumnName
1	FK__Categorie__Paren__4AB81AF0	Categories	ParentID
2	FK__Orders__Customer__4F7CD00D	Orders	CustomerID
3	FK__OrderDeta__Order__52593CB8	OrderDetails	OrderID

CategoryID	ParentID	Name	Lvl	Path

Заполняем CTE



```
USE ShopDB_T12;
GO

INSERT INTO app.Categories (ParentID, Name) VALUES (NULL, 'Электроника');
INSERT INTO app.Categories (ParentID, Name) VALUES (1, 'Компьютеры');
INSERT INTO app.Categories (ParentID, Name) VALUES (2, 'Ноутбуки');
INSERT INTO app.Categories (ParentID, Name) VALUES (3, 'Игровые');
INSERT INTO app.Categories (ParentID, Name) VALUES (4, 'Аксессуары');
GO
```

```
(затронута 1 запись)

(затронута 1 запись)

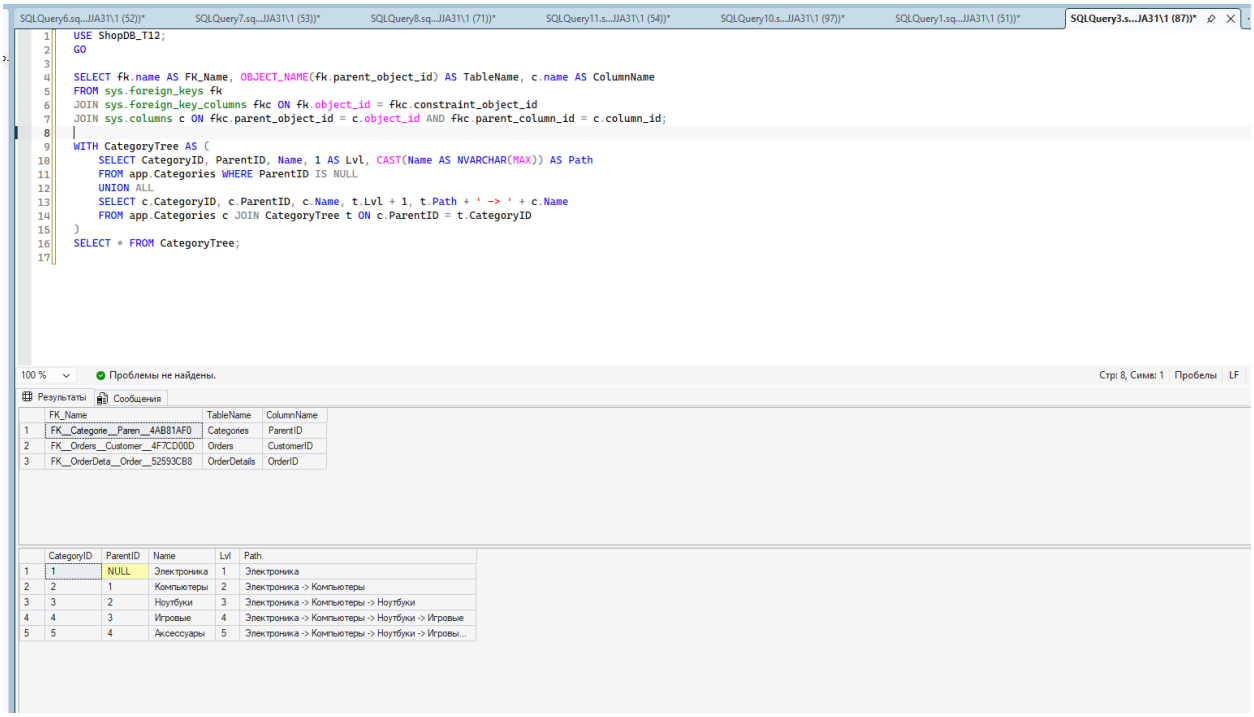
(затронута 1 запись)

(затронута 1 запись)

(затронута 1 запись)

Время выполнения: 2026-10-16T12:39:05.1193897+03:00
```

Уже готовая



```
USE ShopDB_T12;
GO

SELECT fk.name AS FK_Name, OBJECT_NAME(fk.parent_object_id) AS TableName, c.name AS ColumnName
FROM sys.foreign_keys fk
JOIN sys.foreign_key_columns fkc ON fk.object_id = fkc.constraint_object_id
JOIN sys.columns c ON fkc.parent_object_id = c.object_id AND fkc.parent_column_id = c.column_id;

WITH CategoryTree AS (
    SELECT CategoryID, ParentID, Name, 1 AS Lvl, CAST(Name AS NVARCHAR(MAX)) AS Path
    FROM app.Categories WHERE ParentID IS NULL
    UNION ALL
    SELECT c.CategoryID, c.ParentID, c.Name, t.Lvl + 1, t.Path + ' -> ' + c.Name
    FROM app.Categories c JOIN CategoryTree t ON c.ParentID = t.CategoryID
)
SELECT * FROM CategoryTree;
```

	FK_Name	TableName	ColumnName
1	FK__Categorie__Paren__4AB81AF0	Categories	ParentID
2	FK__Orders__Customer__4F7CD00D	Orders	CustomerID
3	FK__OrderDeta__Order__52593CB8	OrderDetails	OrderID

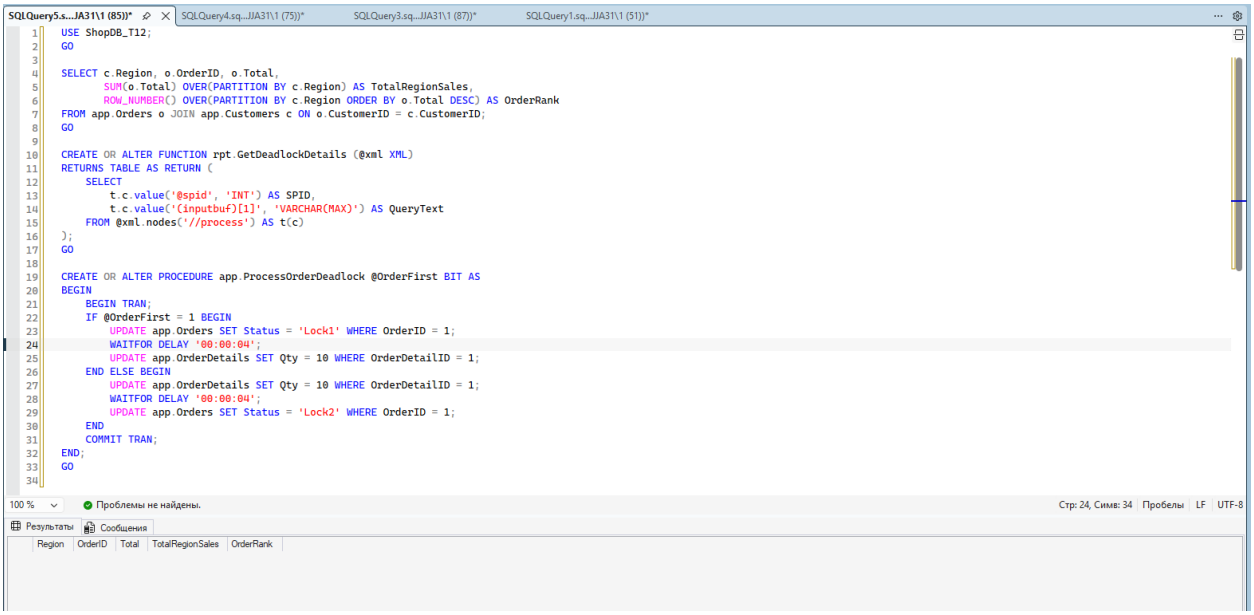
	CategoryID	ParentID	Name	Lvl	Path
1	1	NULL	Электроника	1	Электроника
2	2	1	Компьютеры	2	Электроника -> Компьютеры
3	3	2	Ноутбуки	3	Электроника -> Компьютеры -> Ноутбуки
4	4	3	Игровые	4	Электроника -> Компьютеры -> Ноутбуки -> Игровые
5	5	4	Аксессуары	5	Электроника -> Компьютеры -> Ноутбуки -> Игровы...

То-есть тут я реализовал

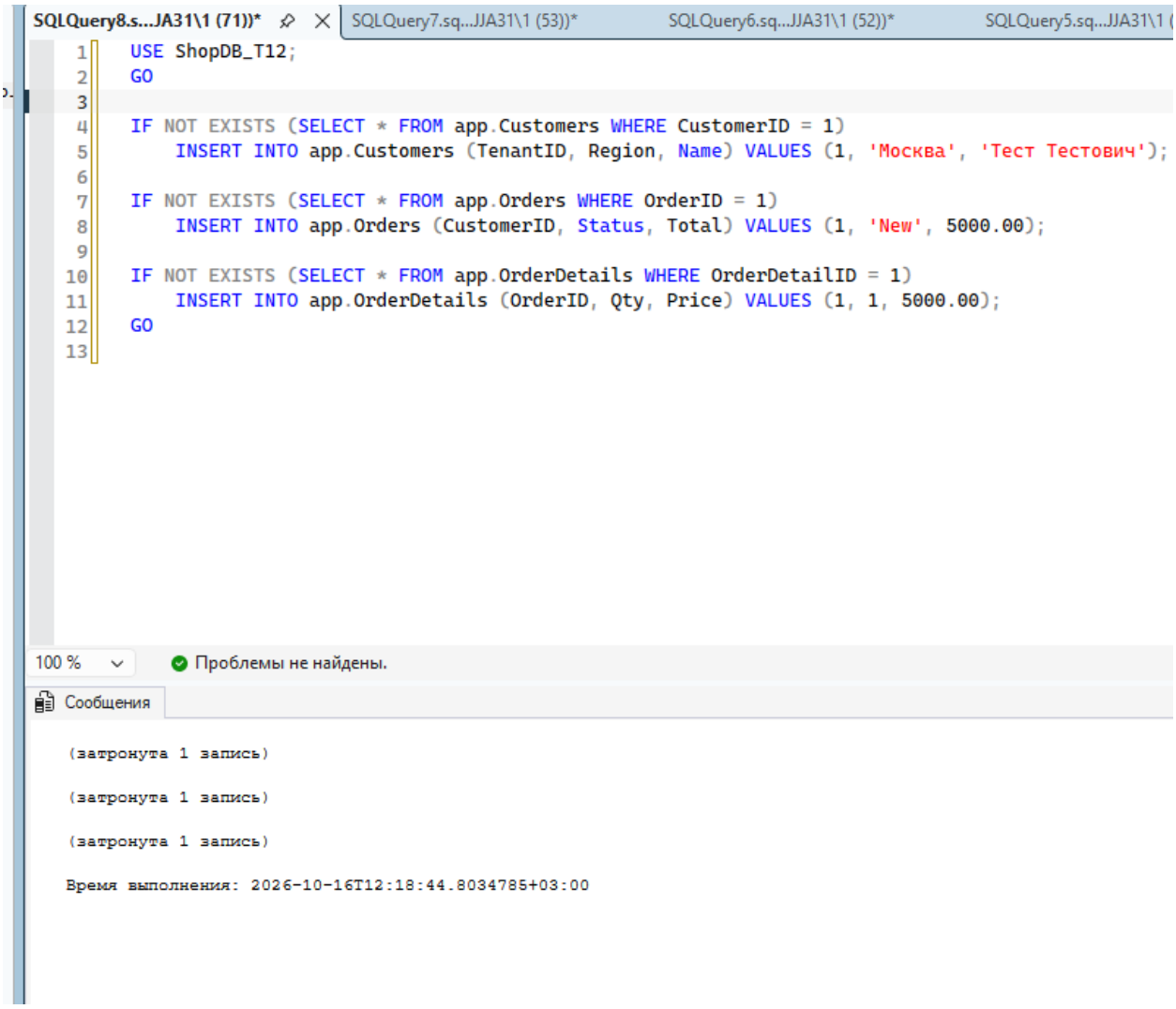
аудит схемы данных, написал запрос к системным представлениям sys.foreign_keys, который уже вывел весь список ключей в базе, благодаря ему, проверили связи между таблицами

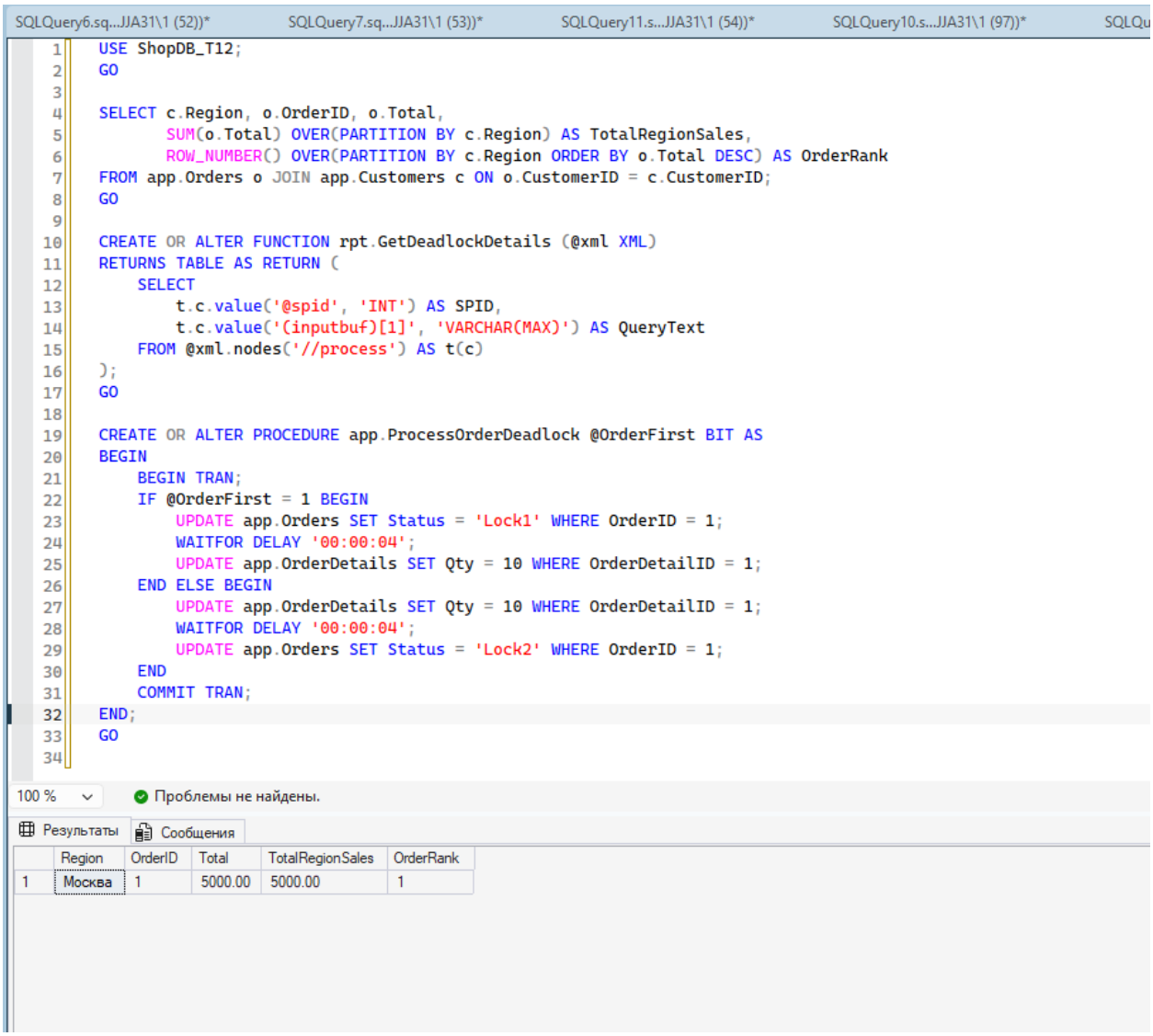
Анализ графа категорий, ну тут я напписал рекурсивное выражение CTE, с автоматическим построением пути Path

2 Задание



добавляем данные

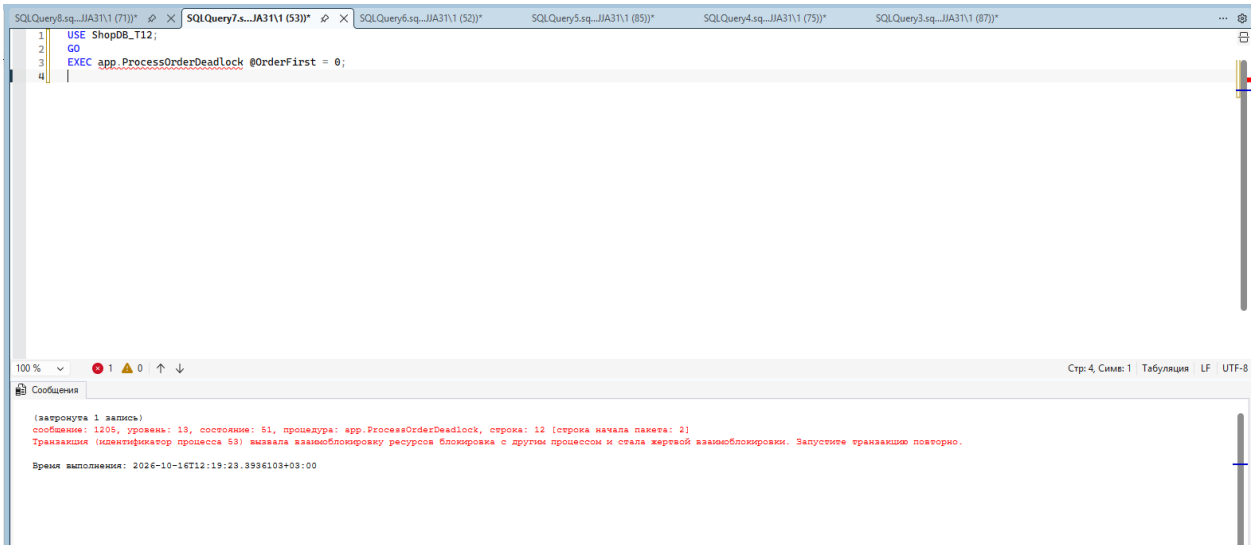

SQLQuery8.s...JJA31\1 (71))*
SQLQuery7.sq...JJA31\1 (53))*
SQLQuery6.sq...JJA31\1 (52))*
SQLQuery5.sq...JJA31\1 (
USE ShopDB_T12;
GO
IF NOT EXISTS (SELECT * FROM app.Customers WHERE CustomerID = 1)
INSERT INTO app.Customers (TenantID, Region, Name) VALUES (1, 'Москва', 'Тест Тестович');
IF NOT EXISTS (SELECT * FROM app.Orders WHERE OrderID = 1)
INSERT INTO app.Orders (CustomerID, Status, Total) VALUES (1, 'New', 5000.00);
IF NOT EXISTS (SELECT * FROM app.OrderDetails WHERE OrderDetailID = 1)
INSERT INTO app.OrderDetails (OrderID, Qty, Price) VALUES (1, 1, 5000.00);
GO
100 %
Проблемы не найдены.
Сообщения
(затронута 1 запись)
(затронута 1 запись)
(затронута 1 запись)
Время выполнения: 2026-10-16T12:18:44.8034785+03:00

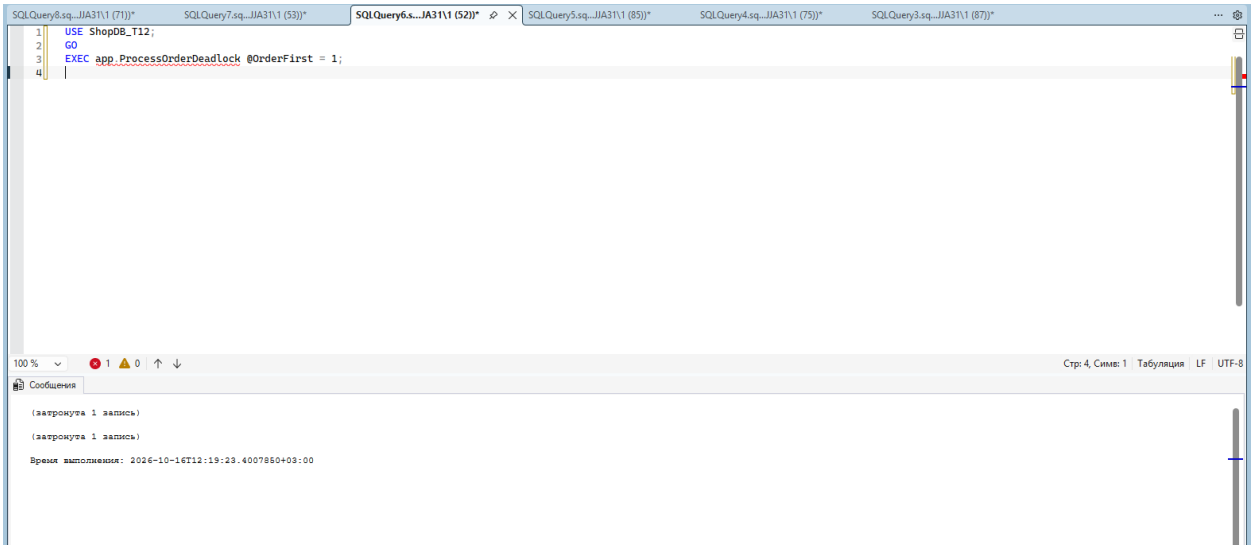


Оконные функции реализовал, написал запрос с sum() over и row_number(), который расчитал сумму продаж по регионам и присвоил заказам ордер
Парсинг XML-графа, создал табличную функцию rpt.GetDeadlockDetails которая с помощью 2 методом .nodes() и .value() преобразует XML отчёт о дедлоке в таблицу

Процедура самого дедлока: написал процедуру app.ProcessOrderDeadlock Она меняет порядок обновлений таблицы Orders. WAITFOR DELAY добавил, она нужна для того, чтобы вызывать взаимоблокировку

3 задание

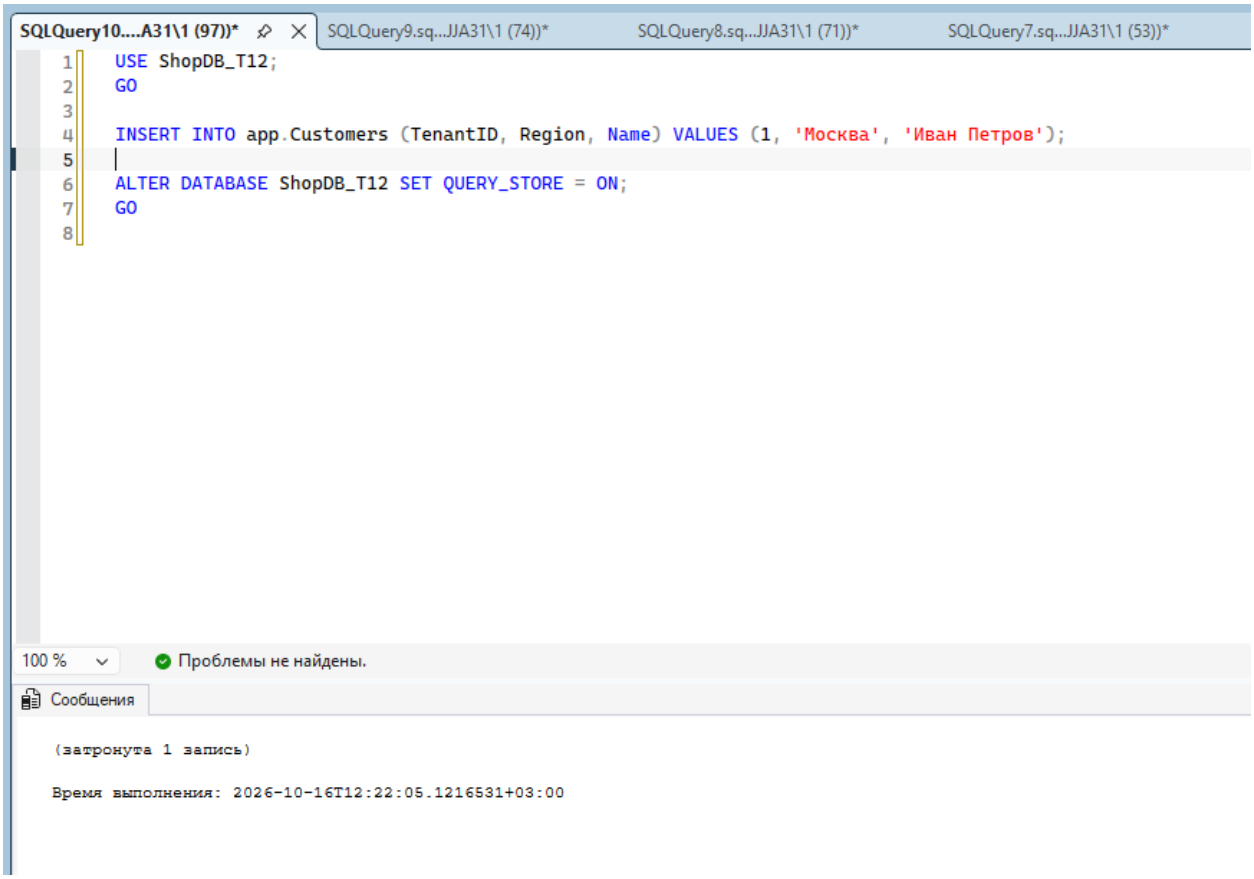




Запустил 2 сессии, я одновременно запустил созданную процедуру в двух новых запросах. Первое окно обновляло таблицы Orders OrderDetails а другая в обратном порядке

Результат Ошибка 1205 из-за разного порядка транзакции заблокировали друг друга

4 задание

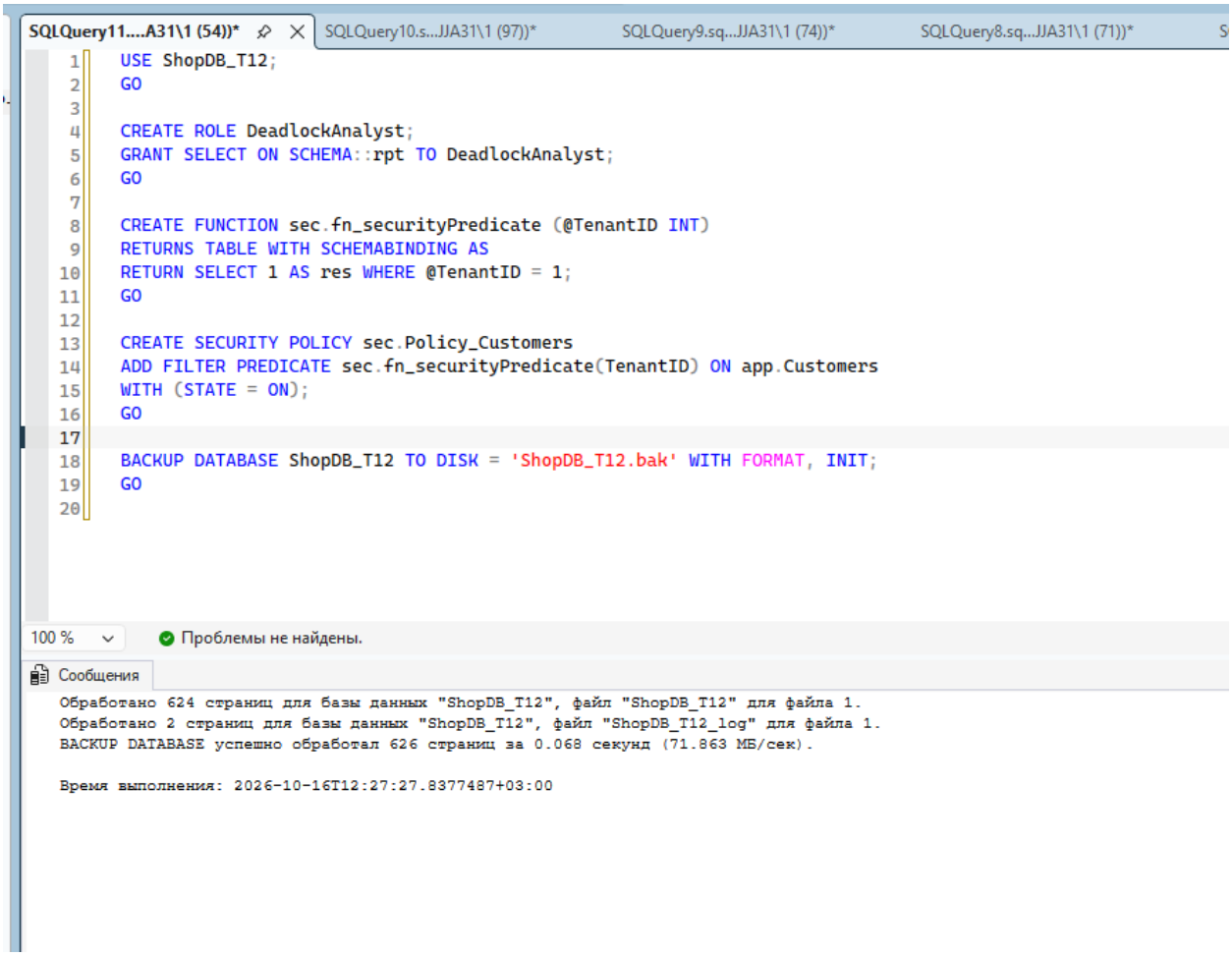


Создал иднекс CustomerID Это перевело блокировки с уровня всей таблицы на конкретные строки типо index seek а потом row lock, теперь они больше не пересикаются

SARGable-зпрос, написал правильный запрос поиска клиента без использования функции в условии where.

Query Store включил в системное хранилище запрос SET QUERY_STORE = ON, которое позволяет находить запросы с наибошьим временем оиждаения

5 задание



Роелвая модель - создал выделенную роль DeadlockAnalyst и выделей минимальные права GRANT SELECT только на схему отчётов rpt для анализа дедлоков

RLS - создал функцию sec.Policy_Customers, которая автоматически филтрует данные клиентов по TenantID

И резервное копирование: успешно выполнил полную копию BACKUP DATABASE